Travail pratique 3 – Sécurité applicative

Objectif

Construire un CTF lié à la sécurité applicative afin de démontrer comment les différentes failles d'une application Web peuvent être exploitées.

Corriger les failles de sécurités en appliquant les bonnes pratiques du développement d'applications Web.

Consignes générales

- Vous devez développer une application composée d'un backend, d'un frontend et de clients automatisés
    - Une application frontend (UI)
    - Une application backend (services + BD)
    - Clients automatisés
        - Simulation d'un client
- Chaque membre de l'équipe doit faire la démonstration du bon fonctionnement de son environnement de développement
    - Réalisé en classe
    - Voir la section « Présentation individuelle de l'infrastructure »
- Chaque membre de l'équipe doit faire la démonstration du bon fonctionnement des 4 premiers flags
    - Réalisé en classe
    - Voir la section « Présentation individuelle des 4 premiers flags »
- Chaque membre de l'équipe doit faire une vidéo démontrant le processus de déploiement et les étapes nécessaires pour atteindre les flags
    - Voir la section « Démonstration du CTF »
- L'utilisation d'intelligences artificielles génératives (ex : ChatGPT, Copilot, etc.): est interdite pour ce travail. Si vous vous dites : « bah, je vais l'utiliser juste pour faire le CSS » et bien je vous dis : « bin faites juste pas de CSS dans ce cas ».
- Travail d'équipe :
    - La note individuelle pourrait être ajusté selon le niveau de contribution au travail d'équipe.
- Responsabilité collective : Chaque membre de l'équipe est responsable de respecter ces consignes. Si un membre ne respecte pas ces règles, toute l'équipe pourrait être pénalisée.

Travail

Ce travail se réalise en équipe de 2 ou 3 (vous n'avez pas à reprendre les mêmes équipes que pour les TP1 et TP2). Chaque équipe doit produire une application Web qui contient des failles que l'on peut exploiter pour obtenir des flags.

Vous n'êtes pas autorisé à copier-coller du code des exemples donnés en classe. Vous pouvez vous en inspirer et même en retranscrire des parties en les modifiant, mais aucun copier-coller.

Contexte

Votre équipe a comme mandat de construire son propre CTF lié à la sécurité applicative. Vous devrez donc mettre sur pied une application web qui a des vulnérabilités ciblées et qu'un attaquant pourra exploiter pour obtenir des flags.

Vous devrez déployer votre infrastructure dans 3 conteneurs Docker qui seront contrôlés par Docker-Compose :

- Application frontend (UI)
  - Le frontend est l'interface graphique de votre application.
  - Vous devez utiliser Python avec Flask
  - Le frontend doit être disponible via le port `15556` de la machine qui roule docker.
  - Le conteneur pour le frontend doit partir de l'image de base[1] :
    - `frgagnon/ctf3-srv:latest` (image `python`)
- Application backend + BD
  - Le backend est un API Rest permettant de faire les opérations.
  - Vous devez utiliser Python avec Flask
  - Vous pouvez mettre la BD dans le même conteneur que votre backend ou dans un autre conteneur si vous préférez.
  - Le backend doit être disponible pour le frontend
  - Le conteneur pour le BackEnd doit partir de l'image de base :
    - `frgagnon/ctf3-srv:latest` (image `python`)
- Simulation d'un usager client
  - Le conteneur pour le client automatisé doit partir de l'image de base :
    - `frgagnon/ctf3-bot:latest` (image `nodejs`)

[1] Vous ne devez pas supprimer les images de base. Vous pouvez ajouter des éléments aux images de base, mais le moins possible pour limiter le temps de création des images. Voir **Annexe 4 –** Information sur les images Docker de base pour savoir ce que ces images contiennent déjà.

Répartition des points

Critères	Pondération	Compétences
Démonstration individuelle de l'environnement (backend, frontend, client automatisé. Capacité à déployer et à vérifier la BD et la FAQ)	15 %	00Q8 3.1, 3.3
Démonstration individuelle des premiers flags (Présentation des vulnérabilités exploitées : Injection SQL, Hashing, XSS, JWT)	15 %	00Q8 3.1, 3.3
Démonstration individuelle en vidéo du CTF[2] (5 à 7 minutes, rigueur, enchainement logique, précision du propos)	30 %	00Q8 1.2, 1.3, 2.4, 3.1, 3.3, 3.4
Failles exploitables et reproductibles par le prof (SQL, Hash, XSS, JWT, Reset pwd, MFA, logique)	10 %	00Q8 2.4, 3.1, 3.3, 3.4
Correctifs appliqués[2] (branche fix, hachage sécurisé, paramétrisation, validation d'entrée, configuration JWT, MFA, gestion de session, cryptographie)	30 %	00Q8.2.4, 3.1, 3.3, 3.4
	100 %	

[2] Au prorata du nombre de failles fonctionnelles

Fonctionnalités de l’application

L’application est une fausse banque dans l’univers de **Lord of the rings**.



Un flag pour les gouverner tous, une faille pour les trouver, un exploit pour les amener tous et dans le dark Web les déverser tous.

1. Un usager non authentifié peut faire trois opérations :
    a. S’authentifier (username/password et dans certains cas MFA)
    b. Faire une demande de réinitialisation de son mot de passe (en fournissant son courriel).
    c. Chercher à partir d’un nom d’usager pour avoir son numéro de compte.
2. Un admin peut simplement :
    a. Créer de nouveaux comptes clients
    b. Le solde initial est de 100$
    c. Un numéro de compte unique (généré automatiquement)
    d. Un nom, un courriel et un mot de passe spécifiés par l’admin au moment de la création.
1. Un client authentifié peut :
    a. Voir son compte, mais pas le compte des autres clients.
    b. Détail du compte:
        i. Un petit message d’accueil (p. ex., Bonjour Gandalf). On utilisera parfois ce message pour y ajouter un flag
        ii. Son nom (pour savoir facilement dans quel compte on se trouve)
        iii. Son numéro de compte
        iv. Son solde
    c. Faire un transfert de son compte vers un autre compte en indiquant le numéro de compte de destination (et le montant).
    d. Voir tous les messages de la FAQ.
    e. Ajouter un message à la FAQ.

Scénarios d'attaque

Votre utilisateur malveillant s'appelle `Saruman`. Il ne possède pas de compte client (encore…), il doit pouvoir réaliser les attaques suivantes :

Injection SQL

Une vulnérabilité aux injections SQL permet d'obtenir la liste de tous les usagers et leur mot de passe de la BD; même pour un usager non authentifié.

Un des noms d'usager sera le flag. Vous devriez minimalement avoir les usagers suivants dans votre BD (à moins d'indication contraire, le MFA ne devrait pas être activé pour ces comptes) :

- `Boromir` (`admin`), solde 0$, mot de **passe faible**.
- `FLAGXYZ` (`utilisateur`), solde 0$, mot de passe solide.
- `Pippin` (`utilisateur`), solde 500$, mot de **passe faible**.
- `Gandalf` (`utilisateur`), solde 500$, mot de passe solide.
- `Arwen` (`utilisateur`), solde 500$, mot de passe solide.
- `Galadriel` (`utilisateur`), solde 500$, mot de **passe faible**, **MFA actif**
- Utilisez le même courriel (un des votres) pour tous ces comptes. Notamment pour Galadriel et Arwen auxquels on enverra des codes. Normalement, nous n'avons pas à recevoir ces courriels pour vrai, mais on veut éviter d'épuiser les limites de notre outil gratuit pour l'envoi de courriel.

Hachage

Les passwords sont stockés de façon non-sécurisée, à la suite de l'attaque précédente, le mot de passe de `Boromir`, `Pippin` et `Galadriel` doivent être suffisamment faibles pour être craqués (à l'aide d'une *rainbow table* disponible en ligne). Il n'y a donc pas, dans la version vulnérable de l'application[3], de mesures en place pour forcer l'utilisation d'un "bon" mot de passe.

Les mots de passe des autres usagers existants doivent être suffisamment complexe pour résister à une attaque *rainbow table* (p. ex., 32 caractères semi-aléatoires avec l'alphabet `a-zA-Z0-9` devraient suffire).

L'attaquant pourra se connecter sur le compte de `Boromir` et pourra se créer son propre compte. L'utilisateur est `Saruman` et l'authentification multi facteurs (MFA) doit être active sur ce compte (valeur par défaut lors de la création d'un compte).

L'attaquant pourra se connecter sur le compte de `Pippin` et il y trouvera un `flag` dans le message de bienvenue de ce compte.

> Du code spécifique au CTF est requis pour rendre les différents flags disponibles (messages d'accueil, JWT, etc.)

[3] Dans la version sécurisée, il devrait y avoir une telle mesure.

Il peut aussi en profiter pour transférer de l'argent dans son propre compte (`Saruman`). Malheureusement, `Pippin` n'a que 500$ dans son compte.

Cross-site scripting (XSS)

En utilisant un compte client valide auquel il a accès (`Pippin` ou `Saruman`), un attaquant pourra poser une question dans la FAQ à titre de client.

Il fera une attaque XSS pour voler les cookies d'un autre client (`Gandalf`).

Dans les cookies de chaque usager, on trouve le JWT associé à l'usager. Mais dans les cookies de `Gandalf` (et non dans les autres), on retrouvera en plus un petit flag.

Pour réaliser cette attaque, vous avez besoin d'un client automatisé qui joue le rôle de `Gandalf` et qui visite régulièrement la FAQ (donc étant authentifié) dans l'application bancaire. Voir le document annexe pour quelques pistes sur la réalisation de ce client.

Si vous n'y arrivez pas, vous pouvez simuler le client automatisé manuellement dans la démo (soyez très clair à ce sujet) pour poursuivre à la prochaine étape.

Vous aurez besoin d'une VM cloud où recevoir le JWT volé à partir du script malicieux injecté.

JSON Web Token (JWT)

Puisque les JWT ne sont pas bien protégés, l'attaquant peut insérer celui trouvé précédemment dans son propre navigateur pour être reconnu comme `Gandalf` à qui le jeton appartient.

Il pourra donc se connecter sur le compte de `Gandalf` et il y trouvera un flag dans le message de bienvenue de ce compte.

L'attaquant peut aussi en profiter pour effectuer un transfert d'argent du compte piraté vers le sien. `Gandalf` n'est pas très riche, il a 500$ dans son compte.

Réinitialisation de mot de passe (Reset password)

Faire un mécanisme de réinitialisation de mot de passe dans lequel un lien est envoyé par courriel avec un code (ou URL) unique.



Mailersend (https://www.mailersend.com) peut vous permettre d'envoyer des courriels[4].
Assurez-vous d'envoyer les courriels à des adresses que vous contrôlés pour éviter de vous faire fermer votre compte!

Le code utilise la même séquence (Utilisation d'un *seed* de départ `s`, puis incrémentation par `+x` à chaque demande de réinitialisation). `s` et `x` changent à chaque lancement de l'application (ça peut être de

[4] Je vous suggère que chaque membre ait son propre compte Mailersend pour éviter les quotas le jour où tout le monde fait sa vidéo…Assurez-vous qu'au moment de la remise, le compte Mailersend qui est commité n'a pas envoyé une grande quantité de courriel pour que ça fonctionne au moment de la correction. Vérifier la limite de Mailersend (e.g., max 500 par mois et 100 par jour).

l'aléatoire faible… le problème ici est le `+x` fixe. On se facilite la vie et on n'a pas à briser l'aléatoire, on a juste à déduire `x`).

Puisque l'attaquant a maintenant un compte, il peut tester cette fonctionnalité sur lui-même pour comprendre la séquence et deviner le prochain code.

Il pourra ensuite déclencher une nouvelle réinitialisation de mot de passe, mais pour le compte d'`Arwen` et utiliser le code connu pour réinitialiser le mot de passe de cet autre usager.

Il pourra donc se connecter sur le compte d'`Arwen` et il y trouvera un flag dans le message de bienvenue de ce compte.

L'attaquant peut aussi en profiter pour effectuer un transfert d'argent du compte piraté vers le sien. `Arwen` a 500$ dans son compte.

Authentification multi facteurs (MFA)

Faire un mécanisme d'authentification multi facteurs, actif sur seulement deux usagers :

- `Saruman`, le nouveau compte que l'attaquant va se créer
- `Galadriel`

Le MFA enverra un code unique par courriel que l'usager devra ensuite entrer (à l'étape suivant l'entrée du mot de passe) pour se connecter.

Le code utilise la même séquence (Utilisation d'un *seed* de départ `s2`, puis incrémentation par `+y` à chaque demande de réinitialisation). `s2` et `y` changent à chaque lancement de l'application.

Comme l'attaquant connaît le mot de passe de `Galadriel`, il pourra donc se connecter sur le compte de `Galadriel` et il y trouvera un flag dans le message de bienvenue de ce compte.

L'attaquant peut aussi en profiter pour effectuer un transfert d'argent du compte piraté vers le sien. `Gladriel` a 500$ dans son compte.

À ce moment, l'attaquant devrait avoir plus de 2000$ dans son compte et peut le revisiter pour voir son 2e flag.

Faille de logique applicative

Ajout d'une faille de logique d'affaire dans l'application.

Pour favoriser l'engagement de ses clients auprès du site, la banque offre une promotion. Chaque jour, un client qui se connecte à son compte peut appuyer sur un bouton pour obtenir 1$ dans son compte.

Après avoir fait l'action pour une journée, le bouton est désactivé pour le client.

Le lendemain, le bouton redeviendra à nouveau accessible par l'usager.

Mais comme il n'y a pas de validation du côté serveur (seulement un bouton désactivé qui empêche), un attaquant peut lui-même déclencher l'appel au serveur pour obtenir plusieurs fois le cadeau. Pour démontrer l'exploitation de cette faille, lorsque le solde de `Saruman` atteindra 2105$, un troisième flag sera affiché dans les messages du compte de cet usager.

Présentation individuelle de l'infrastructure

Individuellement, vous devez faire une démo de votre environnement CTF. La date limite sera indiquée dans le calendrier LEA, mais vous pouvez faire la démo dès que vous êtes prêt.

C'est votre responsabilité de prévoir un moment pour votre démo (à faire pendant les heures de cours).

Lors de la démo, vous devez présenter :

1. L'environnement Docker avec 3 conteneurs (frontend, backend, 1 client automatisé)
    a. Déploiement automatique des conteneurs via scripts (voir remise)
2. L'application web doit avoir au moins la fonctionnalité d'ajouter un message à la FAQ (authentification pas nécessaire à ce moment), autant du côté frontend que backend.
    a. Il n'est pas nécessaire de pouvoir obtenir le contenu de la FAQ via l'application. Lors de la démo, nous irons voir directement sur le conteneur BE le contenu de la BD en mode ligne de commandes (vous devez être en mesure de faire les commandes nécessaires).
3. Le client automatisé (via un script `Puppeteer`) doit ajouter un message à la FAQ dans une boucle infinie à toute les 5 secondes. Chaque message doit être préfixé pas un ID séquentiel (1, 2, 3, 4, 5, 6, ...)

Je vous suggère d'utiliser une branche de code spécifiquement pour la remise partielle afin de pouvoir continuer le développement tout en gardant le nécessaire pour la démo partielle. La branche peut-être la même pour tous les membres de l'équipe.

Vous ne devriez pas utiliser de notes ni utiliser l'historique de vos commandes pour cette démo, l'objectif est de vérifier votre niveau d'aise avec l'infrastructure de votre projet.

Présentation individuelle des 4 premiers flags

Individuellement, vous devez procéder à la démonstration des 4 premiers flags de l'application (injection SQL, hachage, JWT et XSS).

Voici les éléments requis pour la démonstration :

- Explication du vecteur d'attaque
- Réalisation de l'attaque.

Pour la réalisation des attaques vous pouvez utiliser un document de référence contenant les éléments techniques : commandes, extraits de code à utiliser.

On vise une présentation de 5 minutes max.

Démonstration vidéo individuelle du CTF

Chaque personne doit faire une vidéo de présentation dans laquelle elle démontre et explique la réalisation de chacune des attaques. La vidéo ne doit pas nécessairement être une seule séquence, mais la suite doit être logique et sans sauts magiques. **Environ 5 minutes** par vidéo (maximum de 7 minutes).

Considérations pratiques

Le travail sera réalisé en équipe de **2** ou **3** personnes.

Le **TP3** compte pour **20 %** des points de la session. Le **TP3** contient **3 parties**, présentation individuelle de l'infrastructure, démonstration individuelle des 4 premiers flags et le livrable final.

Remise

- **Présentation individuelle de l'application :** Voir la date précise sur LÉA
- **Présentation individuelle des 4 premiers flags :** Voir la date précise sur LÉA
- **Livrable final du projet (rapport, code, vidéo individuelle) :** Voir la date précise sur LÉA

Code du CTF

Votre code doit être consigné sur gitlab Vous devez avoir au moins **2 branches**, une branche `main` contenant la version vulnérable de l'application (le CTF) et une branche `fix` qui contient une copie de master, mais avec les fixes de sécurité appliqués.

Ajoutez des commentaires `#FIX` dans votre code là où vous avez fait les correctifs. Dans cette branche, il doit y avoir un fichier `fix_readme.md` qui indique pour chaque vulnérabilité quels correctifs ont été appliqués et où dans le code (p. ex., `/be/api.py ligne 133`).

- Les 2 branches demandées (`main/master` et `fix`)
- Fichiers `Dockerfile` et `docker compose` nécessaires
- Script de lancement `run.sh`
- Script `stop.sh` qui va fermer les conteneurs, mais aussi surtout supprimer vos images maisons (pas les images de base).

Rapport / démonstration des failles

Une seule remise par équipe.

- Un document PDF qui contient :
  - Page 1 :
    - Le nom des membres de l'équipe
    - L'URL vers votre repo gitlab
      - Donner accès à votre enseignant à votre repo : `@csf_jfdion`
    - Le lien vers la vidéo pour chaque membre
      - Vidéo Youtube non répertorié par exemple
    - Une liste à puce qui indique, pour chacune des 7 attaques demandées, si cette attaque est fonctionnelle à 100% (oui ou non).
  - Page 2 (et+) :
    - Description des étapes pour réaliser chaque attaque dans votre application. C'est votre script pour la démo et le mien pour la correction. Soyez bref, mais **TRÈS** précis (pas de gros texte, une liste à puce idéalement) :
      - `URL`, `routes`, `usernames/password` à utiliser
      - Opérations à faire (séquence)
      - Éléments externes à préparer

Remise

- Les présentations individuelles doivent se faire en classe aussitôt que vous êtes en mesure de démontrer les fonctionnalités demandées
- Sur LÉA une archive zip contenant le document explicatif pour trouver les flags ainsi que les scripts utilisés (partie active) pour y arriver.
- Sur gitlab, un repository partagé avec votre professeur contenant les branches `main/master` et `fix`

> Le nom d'utilisateur gitlab de votre professeur est `frgagnon`

Collaboration et plagiat

Ce travail doit être réalisé uniquement par les membres de l'équipe. Toute aide extérieure, plagiat en partie ou en tout, entrainera la note de 0 % pour ce travail.

Tout étudiant peut être convoqué à une rencontre afin de s'assurer de sa compréhension du travail remis.

Annexe 1 – Client automatisé

Vous devrez simuler un usager qui navigue constamment sur votre application web avec son navigateur Web pour que l'on puisse l'attaquer.

Pour réaliser cette tâche on peut créer un robot automatisé à l'aide de `puppeteer` (librairie javascript) s'exécutant dans un container docker `nodejs`.

Voir la documentation officielle pour un exemple de code

- https://pptr.dev/guides/getting-started

Exemple de robot automatisé

Ce robot se connecte sur une page Web, récupère le titre de la page, puis fait la trace dans la console. Le script se répète aux 5 secondes.

```
import puppeteer from 'puppeteer';

const RUN_INTERVAL = 5000;

const delay = ms => new Promise(res => setTimeout(res, ms));
const run = async () => {
  // configuration de puppeteer pour l'utilisation de chromium
  const browser = await puppeteer.launch({
    headless: true,
    executablePath: '/usr/bin/chromium',
    args: ['--no-sandbox', '--disable-setuid-sandbox']
  });

  // Navigation vers la page, récupération du titre, puis output
  // dans la console
  const page = await browser.newPage();
  console.log("Navigating to web page...");
  await page.goto('http://httpforever');
  await delay(2000);

  const title = await page.title();
  console.log("Title:", title);

  // Utilisation des #id et/ou .class du HTML
  await page.type("#username", "Hank the Tank")
  await page.click("#login")
  await page.waitForNavigation()

  // fermeture de la fençetre du navigateur
  await browser.close();
};

// Lancement du script asynchrone avec une pause de 5 seconde
// entre les exécutions
(async () => {
  while (true) {
    try {
      await run();
    } catch (e) {
      console.error("Error:", e);
    }
    await delay(RUN_INTERVAL);
  }
})();

```

Annexe 2 – Scénario Démonstration individuelle

Pour cette démonstration, vous ne devriez pas utiliser de notes. L'idée est de valider votre niveau de maîtrise des éléments techniques liés à l'infrastructure de votre projet.

1. Les conteneurs ne roulent pas
2. Vérification des bonnes images de base sur les 3 conteneurs
    a. `cat Dockerfile` sur les 3 fichiers de base pour les images
3. Partir les conteneurs à l'aide du script de démarrage
4. Vérifier manuellement (en mode ligne de commande) le contenu de la DB (FAQ):
    a. Se connecter en shell sur le conteneur
    b. Se connecter à la DB pour pouvoir faire des requêtes
    c. Vérifier que la table FAQ est pratiquement vide
5. Se connecter sur l'interface web de votre application dans un navigateur
6. Ajouter un message à la FAQ
7. Re-Vérifier manuellement (directement sur le conteneur) le contenu de la DB (FAQ) :
    a. Présence du nouveau message ajouté via le web
    b. Présence de nouveaux messages ajoutés par le client automatisé

Sur le poste du prof (une seule fois par équipe)

1. Lui transmettre via Teams l'URL pour cloner votre projet en mode `ssh` (`git clone XYZ`).
2. Le prof va lancer votre application à l'aide du script de démarrage (`./run.sh`).
    a. Le lancement est fait dans la VM Linux disponible via LEA.
        i. Le port `25556` de la machine hôte est redirigé vers le port `15556` de la VM; là où devrait rouler votre serveur frontend.
3. Le prof va ensuite se connecter au site web via un navigateur.
    a. Cette manipulation se fait dans la machine hôte (pas dans la VM).
    b. L'application devrait être accessible à partir de `http://127.0.0.1:25556`
4. Le prof va ajouter manuellement un message à la FAQ.
5. Exécution du script `./stop.sh` et vérification des images restantes.

Annexe 3 – Aide-mémoire pour les correctifs (#FIX)

1. Algorithmes et paramètres
    a. Utilisation d'algorithmes recommandés.
    b. Paramétrage documenté et justifié.
2. Protection des données au repos
    a. Chiffrement des éléments sensibles si requis.
3. Nettoyage et validation des entrées
    a. Valider types, longueurs, formats côté serveur.
    b. Rejeter ou normaliser avant traitement.
4. Protection des données à l'affichage
    a. Encodage/échappement des sorties.
5. Accès aux bases de données et SQL
    a. Requêtes paramétrées / ORM.
6. Gestion des mots de passe
    a. Taille et alphabet autorisé.
7. Sessions, JWT et cookies
    a. Protection contre le vol de cookie.
    b. Durée de vie
    c. Révocable
8. Protection contre la force brute et abus
    a. « Rate-limiting » sur login/reset/MFA.
    b. Verrous temporaires si nécessaire.
9. Principe du moindre privilège
    a. Comptes et services avec permissions minimales.
10. Logging, audits
    a. Logs d'audit pour actions sensibles.
11. Secrets et configuration
    a. Variables d'environnement.
    b. Pas de secrets dans le repo.

Annexe 4 – Information sur les images Docker de base

ctf3-srv

```
 GNU nano 6.2                                                Dockerfile
FROM python:3
ADD ./requirements.txt /
RUN pip3 install -r requirements.txt
WORKDIR /mnt/app
```

```
 GNU nano 6.2                                          requirements.txt
Flask
requests
Flask-SQLAlchemy
Flask-bcrypt
bleach
PyJWT
flask-argon2
```

ctf3-bot

```
 GNU nano 6.2                                                Dockerfile
FROM node:22-slim

# Puppeteer prerequisites
RUN apt-get update && apt-get install -y \
    chromium \
    ca-certificates \
    fonts-liberation \
    libasound2 \
    libatk-bridge2.0-0 \
    libatk1.0-0 \
    libc6 \
    libcairo2 \
    libcups2 \
    libdbus-1-3 \
    libexpat1 \
    libfontconfig1 \
    libgbm1 \
    libglib2.0-0 \
    libgtk-3-0 \
    libnspr4 \
    libnss3 \
    libpango-1.0-0 \
    libx11-6 \
    libx11-xcb1 \
    libxcb1 \
    libxcomposite1 \
    libxcursor1 \
    libxdamage1 \
    libxext6 \
    libxfixes3 \
    libxi6 \
    libxrandr2 \
    libxrender1 \
    libxss1 \
    libxtst6 \
    lsb-release \
    xdg-utils \
    wget && \
    apt-get clean && rm -rf /var/lib/apt/lists/*

WORKDIR /app
```

Grille de correction et critères de performance

Critère	A (100 %)	B (85 %)	C (70 %)	D (50 %)	E (0 %)
Démonstration individuelle de l'environnement	Démonstration complète, autonome et rigoureuse. Tous les objectifs de validation sont atteints. Démonstration d'une compréhension globale de l'architecture.	Démonstration fonctionnelle et cohérente. Quelques validations partielles ou ajustements mineurs réalisés avec notes. Capacité à diagnostiquer et corriger les écarts observés.	Démonstration partielle ou nécessitant plusieurs validations externes. Compréhension des objectifs démontrée, mais diagnostic ou corrections imprécises.	Démonstration fragmentée ou non concluante. Difficulté à établir un lien entre actions et objectifs. Aide constante requise.	Aucun environnement fonctionnel ou incapacité à justifier les manipulations effectuées.
Démonstration individuelle des flags	Tous les flags expliqués et reproduits. Chaque attaque est démontrée, justification technique claire.	1 ou 2 flags manquent de détail mineur. Attaques principales fonctionnelles. Rôles et procédure cohérents. Preuves majoritairement complètes.	Seulement 2 flags démontrés correctement. Manque d'uniformité dans les explications ou preuves partielles.	1 seul flag démontré correctement ou plusieurs démonstrations incomplètes/confuses. Difficulté à expliquer étapes. Preuves insuffisantes.	Aucune démonstration, incapacité à reproduire les flags ou justification absente.
Démonstration individuelle en vidéo du CTF	Vidéo claire, structurée, logique. Chaque attaque individuelle expliquée, commandes exécutées, preuves visibles, erreurs anticipées et solutions proposées. Maîtrise technique démontrée.	Vidéo compréhensible et complète sauf petites omissions (ex. un détail de commande). Flux logique respecté. Preuves présentes.	Vidéo fonctionnelle mais désorganisée ou incomplète. Certaines attaques mal détaillées. Preuves partielles.	Vidéo confuse, manquant d'étapes essentielles, l'étudiant ne justifie pas ses choix. Preuves faibles ou absentes.	Vidéo absente, illisible ou hors-sujet.
Failles exploitables	Toutes les vulnérabilités demandées exploitables de façon reproductible (SQLi, hash, XSS, JWT, reset, MFA, logique). Exploits automatisés/scriptés (ex : Cypress) ou procédure manuelle reproductible.	5 ou 6 vulnérabilités exploitables de façon fiable. Les autres sont partiellement exploitables ou nécessitent une simulation claire.	3 ou 4 vulnérabilités exploitables ; les restantes sont incomplètes ou simulées sans justification.	1 ou 2 vulnérabilités exploitables ; la plupart sont manquantes ou non reproductibles.	Aucune vulnérabilité exploitable démontrée.
Correctifs appliqués	Tous les correctifs sont appliqués selon les meilleures pratiques correctement. `fix` contient `#FIX` + `fix_readme.md` détaillant chaque correctif (quoi/ligne/fichier/raison).	Correctifs majoritaires appliqués et documentés. Quelques justifications partielles. Quelques correctifs n'utilisent pas les meilleurs pratiques.	Correctifs appliqués pour certaines failles. Documentation minimale ou incomplète.	Correctifs insuffisants, mal ciblés; documentation absente ou inexistante.	Aucune branche `fix` valable ou correctifs manquants/inefficaces.